

目指せFDE！

ハーネスエンジニアリング入門

AIエージェントを率いて、要件 → 設計 → 開発 → テスト → 監査 → リリースまでを一気通貫で回す
ツール非依存の「原則」と、Claude Code / Codex / Antigravity の実演で学ぶ実践ガイド

対象：開発経験はあるが、AIエージェントの本番運用はこれからの方（FDE志望）

なぜ私がこの講座を作るのか

私は社員8名のソフトウェア開発会社（コールセンター向けの自然言語処理分析ソフトウェアを提供）に飛び込みました。

スクラムチームを2名から8名へ拡大し、7名の営業チームと合わせて15名を率いて、事業計画立案・営業統括・製品企画・R&D／開発・テスト・リリース・プロモーション・営業支援・分析・トライアル・本導入・保守・運用・障害対応までを担いました。

2つの製品で 0→1→10→100 を経験。コールセンターの現場で聞いた“生の声”を製品にフィードバックし、自ら開発して再び現場へ届ける——この仕事に、今のFDEと同じ手応えを感じています。

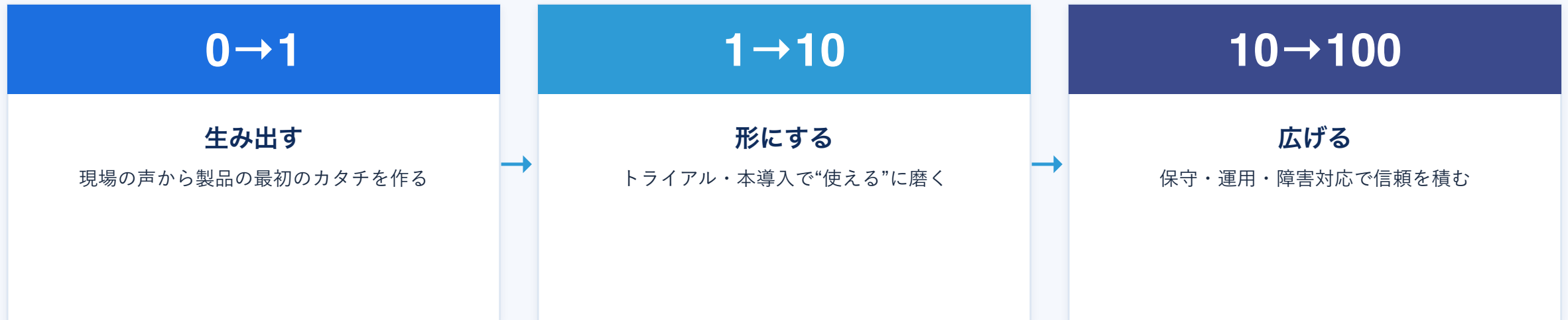
現場で得た確信

現場の声を、動くものへ。

課題の発見から実装・運用までを一人の責任で貫く経験が、ハーネス設計の土台になりました。

この講座は、その実体験から作っています。

0 → 1 → 10 → 100 を経験して分かったこと



各フェーズで必要な“仕組み”は違います。AIエージェント時代も同じ——作るだけでなく、広げ、支え続ける設計こそFDEの価値です。本講座はその全工程をハーネスとして体系化します。

この講座が約束すること

「動くエージェントを1個作る」で終わりません。壊れても自己修復し、監査証跡が残り、本番に安全に出せる“開発ハーネス”の設計力を身につけます。

- 単発のデモではなく、要件からリリースまでの“仕組み”を設計できるようになる
- 特定ツールに依存しない原則を学ぶ（ツールが変わっても価値が残る）
- Claude Code・Codex CLI・Antigravity CLI の3つで同じ作業を体験し、共通点を掴む
- 5本のハンズオンで、手を動かしながら一連の流れを完走する
- ゴールは「AIエージェント群を率いて開発を回せるFDE候補」になること

受講後にできるようになること

1 ハーネスを設計する

オーケストレーション・検証・メモリ・ガードレール・可観測性を意図的に組める

2 要件を仕様に変える

曖昧な要望を、エージェントが正しく実装できる機械可読な仕様へ落とせる

3 品質を保証する

仕様と実装の差分を監査し、メトリクスで“出して良いか”を判断できる

4 安全にリリースする

CI・ガードレール・剥がせる設計で、本番運用まで責任を持って回せる

対象者と前提知識

こんな方に向いています

- 何らかの開発経験がある現役エンジニア
- AIエージェントの“本番運用”はこれからの方
- Cursor等を使うが、属人的で再現性に悩む方
- FDE（Forward Deployed Engineer）を目指す方

前提として必要なもの

- 基本的なコマンドライン操作（cd / ls など）
- Git の基礎（commit / diff / branch）
- いずれかの言語での簡単なコード読解
- ※ 高度なAI/ML知識は不要。原則から学びます

講座マップ：8つのセクション

- Section 0 オリエンテーション — FDEとハーネスの全体像
- Section 1 原則編 — ハーネス設計の5本柱
- Section 2 要件フェーズ — 要望を仕様に変える
- Section 3 設計フェーズ — 制約を先に渡す
- Section 4 自動開発フェーズ — 委譲とガードレール
- Section 5 自動テストフェーズ — テストとeval
- Section 6 監査 / QAフェーズ — 本講座の核
- Section 7 リリースと運用 — 剥がせる設計
- Capstone 要件1枚からリリースまでを完走（ハンズオンは全5本）

開発ライフサイクルの全体像



これが本講座でAIエージェントを率いて回す一本の流れです。要件を仕様に変え、設計で制約を敷き、自動開発で委譲し、自動テストで確かめ、監査/QAで“出して良いか”を判断し、リリースして運用で育て続ける。注目してほしいのは——これは一方通行ではなく、運用で得た学びが要件や設計へ戻る“ループ”だということ。各工程には専用のハーネスが必要で、本講座はそれを一つずつ手渡していきます。今どこを走っているか、いつでもこの図に戻って確認してください。

FDE (Forward Deployed Engineer) とは何か

P POINT

結論

顧客や現場の最前線に入り込み、課題の発見から実装・運用までを一気通貫で担うエンジニアです。

R REASON

理由

AIは「作る」コストを劇的に下げましたが、現場で“成果が出るまで”面倒を見る人が不足しているからです。ビジネス理解と技術力の両方が要ります。

E EXAMPLE

具体例

PalantirがこのロールをつくりOpenAI・Anthropic・Salesforceへ拡大。求人は2025年に約800%増加したと報じられています。

P POINT

再確認

つまりFDEは「技術 × ビジネス × 現場推進」を束ねる希少人材。本講座はその中核スキルを鍛えます。

FDEという仕事の価値（市場データ）

+800%

2025年1～9月のFDE求人増加
（Indeed/FT分析）

700～1500
万
日本での年収目安（経験3～5年～シニア／円）

3要素

Business × Technology × Creativity
の掛け算が必要

数字は調査時点の参考値です。重要なのは「希少性が高く、伸びている職種」だという方向感です。

ハーネス（harness）とは何か

P POINT

結論

AIエージェントの周りに組む“制御装置一式”のこと。エージェント本体ではなく、その外側の仕組みを指します。

R REASON

理由

賢いモデルでも、放置すれば暴走・脱線・同じミスを繰り返します。安全に成果へ導くレール（＝ハーネス）が要るからです。

E EXAMPLE

具体例

馬具(harness)の語源どおり、馬の力を引き出しつつ方向を制御します。テスト・権限・メモリ・ログがその役割です。

P POINT

再確認

ハーネスエンジニアリングとは、この“レール”を設計する技術。FDEの仕事の中心はここにあります。

ハーネスを構成する5つの要素

A オーケストレーション

タスクをどう分割し、どの順で・誰（どのエージェント）に渡すか

B 検証（Verification）

出力が正しいかを自動で確かめる仕組み（テスト・eval・レビュー）

C メモリ

何を覚えさせ、何を毎回渡すか。文脈の設計

D ガードレール

やって良いこと/ダメなことの境界。権限と承認ポイント

E 可観測性

何が起きたかを後から追える。ログ・メトリクス・証跡

なぜ「作れる」だけでは本番で壊れるのか

P POINT

結論

フレームワークでエージェントを動かしても、本番では崩壊しがちです。必要なのは“仕組み”であって“デモ”ではありません。

R REASON

理由

本番は不確実で、入力も状況も変化します。検証・境界・記録がないと、ミスが静かに積み上がり信頼を失うからです。

E EXAMPLE

具体例

「デモは動いたのに、顧客環境で誤動作」「同じバグを何度も再発」——ハーネス不在の現場で必ず起きる典型です。

P POINT

再確認

だから本講座は“ツールの使い方”ではなく“壊れない仕組みの作り方”を学びます。これがFDEの差別化点です。

3つのCLIEージェント、それぞれの得意技

Claude Code

0→1 の初期生成

- 文章・コードのゼロからの生成に強い
- CLAUDE.md・サブエージェント・MCP
- 長時間タスクを粘り強く完遂

Codex CLI

レビュー & 実行検証

- 生成物のレビュー・批判役に強い
- Browser Use / Computer Use
- codex exec で非対話の自動実行

Antigravity CLI

マルチモーダル & 長文脈

- 画像認識などマルチモーダル
- ロングコンテキストで大規模把握
- /goal・/schedule・並列サブエージェント

COLUMN／使い分けの原則

迷ったら：最初に作るのは Claude Code、できたものを点検・実行確認するのは Codex、画像や大量資料を読ませるのは Antigravity。強みを掛け合わせるのがFDE流です。

ハンズオンで使う3つのCLIEージェント

同じ作業を3つのツールで体験します。狙いは比較ではなく「共通する原則」を体に入れること。ツールは変わっても原則は残ります。

- Claude Code (Anthropic) — CLAUDE.md ・サブエージェント ・MCPで長時間タスクに強い
- Codex CLI (OpenAI) — codex で起動、exec でスクリプト化、GPT-5系モデルを選択
- Antigravity CLI (Google) — agy で起動、/goal ・ /schedule、並列サブエージェント対応
- 3つとも「AGENTS.md 的な指示書 + スラッシュコマンド + 権限制御」という同じ骨格を持つ
- ※ コマンドは調査時点のもの。実行前に各公式ドキュメントで最新を確認してください

SECTION 0 — まとめ

「エージェントを作る」から 「エージェントを安全に率いる」へ。

FDEの中核はハーネス設計。次のSectionから、その“5本柱”を一つずつ分解していきます。

01

SECTION

原則編：ハーネス設計の5本柱

ツールに依存しない、エージェントを制御するための5つの土台。ここを押さえれば、どのツールでも応用できます。

ハーネス設計の5本柱（全体像）

1 コンテキスト設計

必要な情報だけを、必要なときにモデルへ渡す

2 制約設計

生成の“前”に解空間を狭める（feedforward）

3 検証ループ

生成の“後”に正しさを確かめ、直す（feedback）

4 メモリ設計

覚えること/忘れることを意図的に決める

5 可観測性

起きたことを測り、追えるようにする

5本柱が支える「信頼できる自動化」

信頼できる自動化（FDEが生み出す価値）

1

コンテキスト
設計

2

制約
設計

3

検証
ループ

4

メモリ
設計

5

可観測性

土台：ツールに依存しない原則 — Claude Code / Codex / Antigravity どれでも応用できる

コンテキストエンジニアリング

P POINT

結論

エージェントの精度は「何を文脈として渡すか」でほぼ決まります。モデルの賢さより、渡す情報の設計が効きます。

R REASON

理由

モデルは渡された文脈の中でしか考えられません。関係ファイル・ルール・制約を整えるほど、ブレない出力になるからです。

E EXAMPLE

具体例

巨大なコードを丸ごと渡すと混乱します。代わりに関連ファイル＋設計ルールだけを選ぶ。MCPやRAGはこれを体系化する道具です。

P POINT

再確認

結論：文脈は“多ければ良い”ではなく“狙って選ぶ”。これが全ての土台になります。

コンテキスト設計の実践ポイント

- プロジェクトの規約・前提を AGENTS.md / CLAUDE.md に1か所へまとめる
- 毎回必要な情報（コーディング規約・ディレクトリ構成）は常駐させる
- タスク固有の情報だけをその都度渡し、無関係なノイズを混ぜない
- MCPで外部データ（DB・Issue・ドキュメント）を必要時だけ接続する
- 長文を貼るより、“どこを見ればよいか”の地図を渡す方が効く

制約設計 (feedforward / 事前制御)

P POINT

結論

生成が始まる“前”に、エージェントが取れる選択肢をあえて狭めます。これが最も費用対効果の高い制御です。

R REASON

理由

後から間違いを直すより、そもそも間違えない範囲に閉じ込める方が安く確実だからです。型やルールが“ガードレール”になります。

E EXAMPLE

具体例

型システム・lint設定・アーキ規約ファイルが好例。「このディレクトリ構成を守れ」「この型を使え」と先に宣言します。

P POINT

再確認

結論：制約は“縛り”ではなく“正解への近道”。先に決めるほど、後が楽になります。

制約設計の実践ポイント

- ルールファイル（規約・禁止事項）を最初に用意する
- 型・スキーマで“ありえない状態”を表現できなくする
- lint / formatter を必須化し、逸脱を自動で弾く
- アーキ境界（依存方向）を明文化してエージェントに渡す
- “やってはいけないこと”を具体例で示すと効果が高い

検証ループ（feedback / 事後制御）

P POINT

結論

生成された結果を自動で確かめ、ダメなら直す——この往復をハーネスに組み込みます。

R REASON

理由

人間が毎回目視するのは続きません。テストやevalが“合否”を返すことで、エージェントは自分で修正できるからです。

E EXAMPLE

具体例

「テスト失敗→原因を読む→修正→再実行」を自動で回す。コンフォーマンス監査で仕様との差分も検証できます。

P POINT

再確認

結論：検証ループの質が、自律性の上限を決めます。確かめられる範囲しか任せられません。

検証ループの実践ポイント

- 「合否が機械で分かる」状態を先に作る（テスト・型・eval）
- エージェントが間違えたら、二度防ぐチェックを1つ足す
- pre-commit / CI で、人手を介さず自動で回す
- 失敗ログをエージェントに渡し、自己修正させる
- 速いフィードバックほど良い（数秒で回るループを狙う）

制約（前）と検証（後） — 生成を挟む両輪

① 制約設計

feedforward／生成の“前”

- 型・スキーマで縛る
- ルール・規約ファイル
- lint で逸脱を自動で弾く
- → そもそも間違えない



② 生成

エージェントが実装

- 狭めた解空間の中で
- コード・仕様を生み出す
- → 枠の中で安全に走る



③ 検証ループ

feedback／生成の“後”

- テスト・eval で合否
- 失敗ログを渡す
- エージェントが自己修正
- → 間違いを直して再生成

ポイント：生成（中央）を、前の制約と後の検証で挟む。前で“狭め”、後で“確かめる”。この両輪が回るほど、安心して任せられる範囲が広がります。

メモリ設計

P POINT

結論

何を覚えさせ、何を毎回忘れさせるか。記憶は“足し算”ではなく“設計”の対象です。

R REASON

理由

全部を覚えさせると文脈が汚れ、コストも精度も悪化します。長期記憶と短期文脈を分けて管理する必要があるからです。

E EXAMPLE

具体例

プロジェクト規約は永続メモリ（ファイル）、今回のタスク文脈は一時的に。決定事項は要約して残す、が定石です。

P POINT

再確認

結論：記憶は資産にも負債にもなる。意図的に“残す/捨てる”を決めるのが設計です。

可観測性 (Observability)

P POINT

結論

エージェントが「何を・なぜ・どうした」を後から追える状態を作ります。見えないものは改善できません。

R REASON

理由

本番で問題が起きたとき、再現と原因特定ができないと信頼を失います。ログと証跡が説明責任を支えるからです。

E EXAMPLE

具体例

実行ログ、変更のdiff、使ったツールの記録、コスト計測。これらが監査証跡となり、リリース判断の材料になります。

P POINT

再確認

結論：可観測性は“保険”ではなく“前提”。測れる現場だけが、安全に自動化を広げられます。

中核思想

エージェントが間違えたら、
二度と同じ間違いをしない“仕組み”を足す。

叱るのでも諦めるのでもなく、ハーネスに1つチェックを加える。この積み重ねが信頼できる自動化を育てます。

作り込みすぎない：剥がせるハーネス（rippable harness）

ハーネスは強力ですが、過剰設計は禁物。モデルが賢くなると、凝った仕組みがかえって足かせになります。

- 良いAGENTS.mdとpre-commitフックは、複雑なミドルウェアより効く
- “いつでも剥がせる”シンプルさを保つ（捨てるやすさ＝強さ）
- モデル更新のたびに、不要になった制約を見直す
- 仕組みを足す前に「本当にこの失敗は繰り返すか？」を問う
- 原則：シンプルに始め、痛みが出た所だけ補強する

SECTION 1 — まとめ

5本柱：コンテキスト・制約・検証・メモリ・可観測性。

次は実際に手を動かします。3つのCLIEージェントを同じ手順でセットアップしましょう。

3つのCLIEージェントをセットアップする

ゴール：3ツールをインストールし、最初の対話とプロジェクト指示書（AGENTS.md）まで用意する

Claude Code

■ インストール

```
$ npm i -g @anthropic-ai/claude-code
```

■ 起動

```
$ claude
```

■ ログイン

ブラウザでAnthropicにサインイン

■ 動作確認

> こんにちは。今のフォルダを説明して

Codex CLI

■ インストール

```
$ npm i -g @openai/codex  
(または curl 経由のスクリプト)
```

■ 起動

```
$ codex
```

■ ログイン

ChatGPTアカウント or APIキー

■ 動作確認

> このリポジトリの構成を教えてください

Antigravity CLI

■ インストール

```
$ curl -fsSL \  
https://antigravity.google/cli/install.sh | bash
```

■ 起動

```
$ agy
```

■ ログイン

Googleアカウントで認証

■ 動作確認

> /goal プロジェクト概要を把握する

プロジェクト指示書（AGENTS.md）を用意する

ゴール：3ツール共通の骨格——指示書・スラッシュコマンド・権限——を体感する

Claude Code

■ 指示書を生成

```
$ claude
```

```
> /init
```

→ CLAUDE.md が作られる

■ 中身を編集

規約・構成・禁止事項を記入

■ 便利コマンド

```
/agents (サブエージェント)
```

```
/mcp (外部接続)
```

Codex CLI

■ 指示書

AGENTS.md を手で作成

（規約・前提・制約を記述）

■ モデル選択

```
> /model
```

→ GPT-5系を切替

■ スクリプト化

```
$ codex exec "テストを実行して"
```

（非対話で自動実行）

Antigravity CLI

■ 指示書

AGENTS.md を作成

（Claude/Codexと共通の考え方）

■ ゴール駆動

```
> /goal ○○を実装する
```

■ スケジュール

```
> /schedule 毎朝lintを回す
```

■ 並列

サブエージェントを並列起動

チェックポイントとつまずき対処

- 3ツールとも「指示書 + 対話 + 権限制御」という同じ骨格だと確認できたか
- 起動できない → Node.jsのバージョン、PATH、権限を確認
- AGENTS.md / CLAUDE.md に最低限「規約・構成・禁止事項」を書けたか
- いきなり大改修させず、まず“読む・説明させる”から始められたか
- 気づき：ツールが違ってても、人間がやるべき準備は同じ

02

SECTION

要件フェーズ：要望を仕様に変える

曖昧な「こうしたい」を、エージェントが正しく実装できる“機械可読な仕様”へ翻訳します。

要件フェーズをエージェントに任せる

P POINT

結論

要件定義こそ、エージェントとの協働で最も価値が出る工程です。コードより先に“仕様”を固めます。

R REASON

理由

実装の失敗の多くは「指示が曖昧だった」ことが原因。最初に仕様を機械可読にすれば、後の全工程が安定するからです。

E EXAMPLE

具体例

自然言語の要望をエージェントに渡し、仕様書（spec）として構造化させる。Spec-Driven Developmentの出発点です。

P POINT

再確認

結論：良い仕様は最良のハーネス。要件フェーズの質が、プロジェクト全体の質を決めます。

曖昧な要望 → 機械可読な仕様へ

人間の「いい感じにして」は、エージェントには通じません。検証可能な形まで具体化します。

- 目的・利用者・成功条件（受け入れ基準）を明記する
- 入力と出力、エッジケースを列挙する
- やらないこと（スコープ外）も書く
- “どうなったら完成か”をテスト可能な文で表現する
- 仕様はファイルにして、変更履歴をGitで追えるようにする

Spec-Driven Development（仕様駆動開発）とは

P POINT

結論

コードの前に仕様を書き、仕様を“正”として開発・検証を進める手法です。

R REASON

理由

仕様が中心にあれば、実装も検証もテストもそこから導けます。要件→仕様→実装の監査証跡（diff）も残るからです。

E EXAMPLE

具体例

GitHub Spec Kit・AWS Kiro・Claude Code・BMAD など各社が独自のSDDを2026年に提供。共通点は“仕様が起点”です。

P POINT

再確認

結論：SDDは「仕様＝ハーネスの中核」という思想。本講座の全工程がこの上に乗ります。

良い仕様書の条件

1 曖昧さがない

解釈が一通りに定まる。「速い」ではなく「200ms以内」

2 検証可能

完成条件が、テストやチェックで合否判定できる

3 スcopeが明確

やること/やらないことの境界がはっきりしている

4 文脈を含む

前提・制約・既存仕様への参照がある

仕様の監査証跡（diffベース）

要件が変わったら、仕様を“差分”で更新します。これが「いつ・なぜ変わったか」の証跡になります。

- 要件文＋既存仕様をエージェントに渡し、変更diffを出させる
- diffをレビュー → 承認 → コミット、で履歴が残る
- “黒箱で勝手に変わった”を防ぎ、説明責任を果たせる
- 規制業界では、この証跡そのものが価値になる
- 原則：仕様変更も、コード変更と同じ規律で管理する

要件フェーズのエージェント役割分担（例）

A アナリスト役

要望をヒアリングし、曖昧点を質問で潰す

P PM役

PRD（要件定義書）にまとめ、優先度をつける

R レビューアー役

仕様の抜け・矛盾・検証不能な記述を指摘する

悪い指示 と 良い指示

× 悪い指示

- 「ログイン機能をいい感じに作って」
- 成功条件が書かれていない
- エラー時やエッジケースが不明
- → エージェントが推測で暴走する

○ 良い指示

- 「メール+パスワードでログイン。5回失敗で15分ロック」
- 受け入れ基準をテスト可能に明記
- 異常系（無効入力・期限切れ）も定義
- → 検証でき、自己修正もできる

SECTION 2 — まとめ

コードの前に、検証できる仕様を。

仕様が機械可読なら、後の開発・テスト・監査はすべてそこから導けます。次はハンズオンで仕様を作ります。

要望から仕様書を生成する

ゴール：1枚の要望メモから、受け入れ基準付きの spec.md をエージェントに作らせる

Claude Code

- 要望を渡す
 - > 要望メモを基に spec.md を作成。
目的/利用者/受け入れ基準/
スコープ外/異常系を含めて
- 反復
 - > 受け入れ基準をテスト可能に書き直して
- 保存
 - > spec.md として保存

Codex CLI

- 要望を渡す
 - \$ codex
 - > requirements.txt から
仕様 spec.md を生成して
- モデル調整
 - > /model (推論レベルを上げる)
- 非対話生成
 - \$ codex exec \
"spec.md を更新"

Antigravity CLI

- ゴール設定
 - > /goal 要望を仕様書に変換
- 生成
 - > spec.md を作成。曖昧な点は
質問してから書いて
- 並列確認
 - サブエージェントに
矛盾チェックを並列で依頼

仕様をレビューし、差分で更新する

ゴール：生成された仕様の穴をレビューし、変更を diff として残す習慣をつくる

Claude Code

■ レビュー依頼

> `spec.md` の曖昧点・検証不能な
記述を指摘して

■ 修正を差分で

> 指摘箇所だけ修正案を diff で

■ 記録

```
$ git add spec.md
$ git commit -m "spec: 受け入れ基準を明確化"
```

Codex CLI

■ レビュー依頼

> `spec.md` の抜け漏れを列挙して

■ 反映

> 承認した項目だけ反映して

■ 確認

```
$ git diff spec.md
```

■ コミット

```
$ git commit -am "spec更新"
```

Antigravity CLI

■ レビュー

> `/goal` 仕様の矛盾を洗い出す

■ 反映

> 合意した変更のみ適用

■ 証跡

変更理由をコミットメッセージへ

```
$ git commit -am \
"spec: 異常系を追加"
```

チェックポイント

- 完成した spec.md の「受け入れ基準」は、テストに落とせる粒度か
- スコープ外・異常系まで書けているか（ここを抜くと後で揉める）
- 仕様変更が git diff で追える状態になっているか
- エージェントに“質問させて”から書かせると精度が上がると体感できたか
- 気づき：仕様はこの後の全工程の“契約書”になる

03

SECTION

設計フェーズ：制約を先に渡す

アーキテクチャの制約を“生成の前”にエージェントへ渡し、解空間を正解の近くに狭めます。

システム設計フェーズの考え方

P POINT

結論

設計とは、エージェントが守るべき“地形（制約）”を先に敷くことです。自由にさせるほど脱線します。

R REASON

理由

制約を後出しすると手戻りが増えます。型・ルール・境界を先に渡せば、出力が最初から枠の中に収まるからです（柱2の応用）。

E EXAMPLE

具体例

「依存はこの方向だけ」「この層をまたぐな」「この型を使い」を設計ドキュメントとして渡し、実装前に合意します。

P POINT

再確認

結論：設計フェーズは“制約を仕込む”工程。ここで敷いたルールが、開発フェーズの暴走を防ぎます。

アーキ制約を事前に渡す（ルール / 型 / lint）

制約は文章で説教するより、機械が強制できる形にするほど効きます。

- **ルールファイル**：命名・構成・禁止パターンを明文化
- **型・スキーマ**：不正な状態を“表現できなく”する
- **lintルール**：アーキ違反（依存方向など）を自動検出
- **設計ドキュメント**：図と意図をエージェントの文脈に入れる
- **原則**：人間が見張るのではなく、仕組みに見張らせる

制約を仕込む3つの場所

1 ルール（言葉）

AGENTS.md等に規約・禁止事項。読ませて守らせる

2 型（構造）

型・スキーマで不正を“コンパイル時に”弾く

3 lint/CI（自動）

逸脱を機械が検出し、マージ前に止める

設計レビューをエージェントと回す

P POINT

結論

設計案そのものを、別のエージェント（または別ロール）にレビューさせます。

R REASON

理由

一人（一エージェント）の設計は盲点が残ります。批判役を分けると、矛盾や見落としが早期に見つかるからです。

E EXAMPLE

具体例

「この設計のリスクと代替案を3つ挙げて」と問う。アーキ役と批判役を分け、合意した設計だけを実装に進めます。

P POINT

再確認

結論：設計は“生成”だけでなく“相互レビュー”で固める。人間は最終承認に集中できます。

ブラックボックス生成を避ける段階制御

「全部一気に作って」は危険。計画→承認→小さく実装、と刻むことで“黒箱”を防ぎます。

- まず計画（plan）を出させ、人間が承認してから着手
- 大きな変更を、レビュー可能な小さなステップに分割
- 各ステップで diff を確認し、意図と違えば即修正
- “なぜそうしたか”を説明させ、推測の暴走を止める
- 原則：自律性は上げるが、見える化は手放さない

一括生成 と 段階制御

× 一括生成（黒箱）

- 「全部作って」と丸投げ
- 大量のdiffが一度に出てレビュー不能
- 間違いの原因特定が困難
- → 後戻りコストが爆発する

○ 段階制御

- 計画 → 承認 → 小さく実装 → 検証
- 各ステップの diff が読める
- ずれたらその場で軌道修正
- → 速度と安全を両立できる

SECTION 3 — まとめ

自由に作らせる前に、レールを敷く。

型・ルール・lintで制約を先に渡し、計画→承認→実装で刻む。次はハンズオンで制約ファイルから設計します。

制約ファイルを作り、設計させる

ゴール：アーキ制約を明文化し、エージェントに“制約に従った設計案”を出させる

Claude Code

■ 制約を記述

CLAUDE.md に依存方向・層・

禁止パターンを追記

■ 設計依頼

> 制約に従い設計案を出して。

リスクと代替案も3つ

■ レビュー役

> /agents で批判役を作り点検

Codex CLI

■ 制約を記述

AGENTS.md に規約を追記

■ 設計依頼

\$ codex

> spec.md と制約に基づき

設計を提案して

■ 確認

> 各判断の理由を説明して

Antigravity CLI

■ 制約を記述

AGENTS.md に制約を追記

■ 設計依頼

> /goal 制約準拠の設計を作る

■ 並列レビュー

サブエージェントに

矛盾・リスク点検を並列依頼

計画 → 承認 → 小さく実装

ゴール：黒箱を避け、計画を承認してから最小ステップで実装する流れを身につける

Claude Code

■ 計画

> まず実装計画を箇条書きで。

コードはまだ書かないで

■ 承認後に着手

> ステップ1だけ実装して

■ 差分確認

```
$ git diff
```

> 意図と違う点を修正

Codex CLI

■ 計画

> 実装プランを提示して

(着手はしない)

■ 段階実装

> 承認した1項目だけ実装

■ 確認

```
$ git diff
```

```
$ codex exec "テスト実行"
```

Antigravity CLI

■ 計画

> /goal 実装計画を作る

(承認まで実装しない)

■ 段階実装

> 1ステップずつ進めて

■ 確認

各ステップでdiffを提示させる

チェックポイント

- 制約を“言葉＋機械チェック”の両方で渡せたか
- 設計案に対し、リスクと代替案を引き出せたか
- 計画を承認してから実装に入る流れを守れたか
- 1ステップごとの diff をレビューできる大きさに保てたか
- 気づき：制約を先に渡すほど、生成がブレない

04

SECTION

自動開発フェーズ：委譲とガードレール

タスクを分割してエージェントに委ね、権限と承認ポイントで安全に走らせます。

自動開発フェーズの全体像

P POINT

結論

実装はエージェントに委譲しつつ、人間は“境界の番人”として要所だけ握ります。

R REASON

理由

全部を人がやれば自動化の意味がなく、全部任せれば暴走します。委譲と統制のバランス設計が要るからです。

E EXAMPLE

具体例

タスクを小さく分け、サブエージェントに割り当て、危険な操作だけ人間の承認を挟む——という構成を組みます。

P POINT

再確認

結論：自動開発とは“放任”ではなく“委譲設計”。誰に何を任せ、どこで止めるかを決める仕事です。

タスク分割と委譲

エージェントは、小さく明確なタスクほど高い精度を出します。分割の質が成果を左右します。

- **1タスク=1つの検証可能な成果（テストが書ける単位）**
- 依存関係を意識し、並列にできる所は並列で進める
- 各タスクに「完了の定義」を添える
- 大きすぎるタスクは、エージェント自身に分割案を出させる
- 原則：レビューできる粒度を超えて任せない

サブエージェント / マルチロール構成

P POINT

結論

1体に全部やらせず、役割の違うエージェントを複数立てて分業させます。

R REASON

理由

役割を分けると各自の文脈が小さく保て、相互チェックも働きます。実装役と検証役を分けると品質が上がるからです。

E EXAMPLE

具体例

BMADでは アナリスト・PM・アーキテクト・開発・QA など名前付きの役割に分業。Claude Codeのサブエージェントも同様の発想です。

P POINT

再確認

結論：マルチロールは“小さな組織”。FDEはその組織を設計し、指揮する立場になります。

マルチロール構成の例

1 アーキテクト役

設計と制約を維持し、全体の整合性を見る

2 開発役

タスク単位で実装する。小さく刻む

3 QA役

テストとevalで検証し、合否を返す

4 レビュー役

diffと設計逸脱を点検し、指摘する

権限とガードレール

P POINT

結論

エージェントに与える“できること”を最小限に絞り、危険操作には承認を挟みます。

R REASON

理由

ファイル削除・本番デプロイ・課金APIなどは、誤爆の被害が大きいためです。最小権限はセキュリティの基本でもあります。

E EXAMPLE

具体例

読み取りは自由、書き込みは確認、破壊的操作（削除・push）は明示承認、という段階的な権限設計が定石です。

P POINT

再確認

結論：自律性とガードレールはセットで設計する。自由度は“安全に戻せる範囲”で上げます。

human-in-the-loop（人間の介入点）の置き所

全部を確認すると遅く、確認ゼロは危険。介入点を“効く場所”に絞ります。

- 破壊的操作（削除・本番反映・外部送信）の直前は必ず人間
- 設計の方向を決める分岐点でレビューを挟む
- ルーチンで低リスクな作業は自動で通す
- 失敗時に自動で止まり、人へエスカレーションする経路を用意
- 原則：介入は“量”でなく“位置”で効かせる

開発を加速する“小さなハーネス”

1 AGENTS.md / CLAUDE.md

規約・制約の常駐。最も費用対効果が高い

2 pre-commitフック

コミット前にlint/テストを自動実行

3 CIパイプライン

マージ前に検証を強制し、逸脱を止める

SECTION 4 — まとめ

委譲は設計。誰に・何を・どこまで任せるか。

小さく分け、役割で分業し、危険操作に人を置く。次は“任せた結果が正しいか”を確かめるテストへ。

05

SECTION

自動テストフェーズ：テストとeval

エージェントの出力が正しいかを自動で確かめ、間違いを検証ループで潰します。

自動テストフェーズの役割

P POINT

結論

テストは“品質チェック”であると同時に、エージェントの“自己修正の燃料”です。

R REASON

理由

合否が機械で返るほど、エージェントは人手なしで直せます。検証ループ（柱3）の質が自律性の上限を決めるからです。

E EXAMPLE

具体例

テスト戦略の立案・テストコード生成・実行・失敗解析まで、エージェントに任せられる範囲を広げていきます。

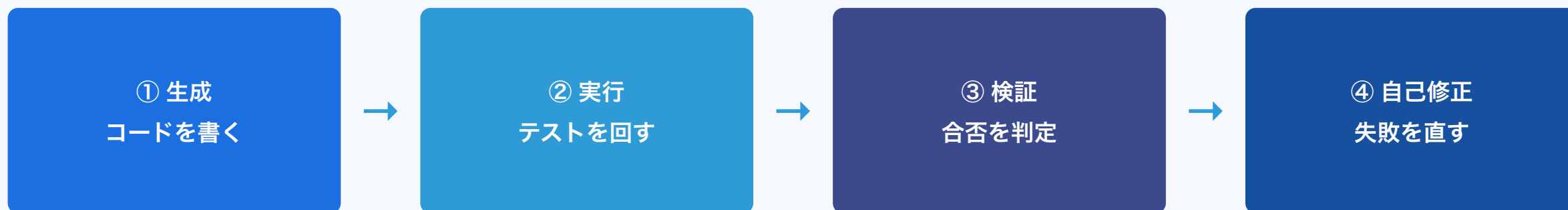
P POINT

再確認

結論：テストは“後工程”ではなく“ハーネスの心臓”。確かめられる範囲だけ、安全に自動化できます。

検証ループ：生成 → 実行 → 検証 → 自己修正

🔄 ループは速いほど良い — 失敗ログを渡せば、エージェントが自分で直す



③で失敗したら、ログを④へ → 直して①へ戻る。このループが速く・自動で回るほど、人手は減り、任せられる範囲は広がる。テストとevalは、このループを駆動するエンジンです。

エージェントによるテスト戦略と生成

「とりあえずテストを書いて」ではなく、何をどう守るかの戦略から組ませます。

- 受け入れ基準（spec）から逆算してテストを設計させる
- 正常系だけでなく、異常系・境界値を網羅させる
- 壊れやすい所・重要な所に厚くテストを置く
- 生成したテストは人間がサンプルレビューする
- 原則：テストは仕様の“実行可能な写し”である

evals : エージェントの品質を測る

P POINT

結論

通常のテストに加え、エージェントの“振る舞い”を評価する仕組みがevalsです。

R REASON

理由

AIの出力は毎回少しずつ変わります。固定の正解だけでなく、品質を継続的に採点する仕組みが要るからです。

E EXAMPLE

具体例

代表的な入力セットに対し、期待する性質（正確さ・形式・安全性）を自動採点。回帰（劣化）を検知できます。

P POINT

再確認

結論：testは“壊れていないか”、evalは“賢く振る舞えているか”。両輪で品質を支えます。

テストの3層

1 ユニットテスト

個々の関数・部品が正しいか。速くて土台

2 結合テスト

部品をつないだ動作。仕様の受け入れ基準に対応

3 eval

エージェントの振る舞いを採点。回帰を検知

回帰を検知し、検証ループに組み込む

一度直したバグを二度と再発させない——これがハーネスの中核思想の実装です。

- **バグを直したら、その再発を防ぐテストを必ず1本足す**
- テスト・evalを pre-commit / CI に組み込み自動実行
- 失敗ログをエージェントへ渡し、自己修正させる
- ループは速いほど良い（数秒～数分で回す）
- 原則：間違えたら、仕組みに1つチェックを追加する

テストなし運用 と テスト付き運用

× テストなし

- 出力が正しいか毎回人が目視
- 同じバグが何度も再発
- エージェントは自己修正できない
- → 自律性を上げられない

○ テスト + eval 付き

- 合否が機械で即座に返る
- 再発はテストが先回りで防ぐ
- 失敗を渡せば自分で直す
- → 安心して任せられる範囲が広がる

SECTION 5 — まとめ

確かめられる範囲しか、任せられない。

test = 壊れていないか、eval = 賢く振る舞えているか。次のハンズオンで両方を回します。

テストを生成して実行する

ゴール：spec の受け入れ基準から、正常系・異常系のテストを生成し実行する

Claude Code

- 生成
 - > spec.mdの受け入れ基準から
テストを作成。異常系も
- 実行
 - > テストを実行して
- 自己修正
 - > 失敗を読み、原因を直して
再実行して

Codex CLI

- 生成
 - \$ codex
 - > 受け入れ基準を網羅する

テストを書いて
- 実行(非対話)
 - \$ codex exec \
"テストを実行し失敗を修正"

Antigravity CLI

- 生成
 - > /goal 受け入れ基準のテスト整備
- 実行
 - > テストを実行して
- 並列
 - サブエージェントに
境界値ケース追加を並列依頼

eval を回し、回帰を防ぐ

ゴール：代表入力セットで振る舞いを採点し、再発防止テストを足してCIに組み込む

Claude Code

- eval設計
 - > 代表入力と期待性質を定義し
簡単なeval scriptを作って
- 回帰防止
 - > 直したバグの再発テストを追加
- 自動化
 - pre-commitにテストを登録

Codex CLI

- eval
 - > 入力例集に対する採点を実装
- 確認
 - \$ `codex exec "evalを実行"`
- 自動化
 - CI設定にテスト/evalを追加
(GitHub Actions等)

Antigravity CLI

- eval
 - > `/goal eval`を整備して回す
- スケジュール
 - > `/schedule` 毎朝evalを実行
- 回帰防止
 - 再発防止テストを追加して
commit

チェックポイント

- 受け入れ基準とテストが1対1で対応づいているか
- 異常系・境界値まで生成できたか
- 失敗ログを渡したらエージェントが自己修正できたか
- テスト/evalが pre-commit や CI で自動実行されるか
- 気づき：検証の自動化が、任せられる範囲を広げる

06

SECTION

監査 / QAフェーズ（本講座の核）

“出して良いか”を判断する工程。仕様と実装の差分を監査し、メトリクスで品質を保証します。

監査 / QAフェーズが差別化の核である理由

P POINT

結論

多くの講座が素通りするこの工程こそ、FDEが信頼を勝ち取る決め所です。

R REASON

理由

本番投入の可否を、勘ではなく証拠で判断できるかが、現場での信頼を分けるからです。ここが弱いと自動化は広がりません。

E EXAMPLE

具体例

仕様との整合性チェック、監査証跡、品質メトリクスを揃え、「リリースして良い」と説明できる状態を作ります。

P POINT

再確認

結論：監査/QAは“仕上げ”ではなく“信頼の生産工程”。ここを厚くするのが本講座の戦略です。

パフォーマンス監査：仕様 vs 実装

P POINT

結論

「仕様どおりに作られているか」を、エージェントに自動照合させます。

R REASON

理由

コードと仕様は時間とともにズレます。差分を機械的に洗い出せば、抜けや余計な実装を早く見つけれられるからです。

E EXAMPLE

具体例

コードにアクセスできるエージェントが、仕様にあって未実装の箇所／実装にあって仕様がない箇所をレポート化します。

P POINT

再確認

結論：監査は“目視”から“自動照合”へ。仕様という基準があるから、ズレを定量化できます。

監査証跡と規制業界テンプレート

「誰が・いつ・なぜ・何を変えたか」を残すこと自体が、規制下では価値になります。

- 要件→仕様→実装→テストの変更履歴をdiffで一貫して残す
- 判断の根拠（なぜ承認したか）をコミットや記録に残す
- Tesslerのように監査証跡・規制テンプレ・CI連携を備えたツールもある
- 金融・医療など規制業界ではこの証跡が必須になる
- 原則：説明できない自動化は、本番に出せない

追うべき品質メトリクス（新規計装ゼロで始められる）

1 マージPRあたりコスト

1つの成果を出すのにいくらかったか（\$/PR）

2 time-to-merge

エージェント支援PRが取り込まれるまでの時間

3 レビュー速度 vs PRサイズ

PRの大きさに対するレビューの捗り

4 開発者あたりcompute支出

アクティブ開発者ごとの計算コスト

メトリクスで“見える化”する（考え方の例）

\$ / PR

コストが見えれば、どのハーネス層に
投資すべきか分かる

時間

time-to-merge短縮は、ループが速い
証拠

0計装

既存のGit/CIデータから今日すぐ測り
始められる

ポイントは“完璧な計測”より“まず測って判断材料にする”こと。数値は各現場の実測で置き換えてください。

品質ゲートとリリース判断

P POINT

結論

「全テスト合格・監査差分ゼロ・メトリクス基準内」を満たして初めてリリース可、というゲートを設けます。

R REASON

理由

人の気分でリリースすると品質がブレます。明文化したゲートがあれば、誰が見ても同じ判断ができるからです。

E EXAMPLE

具体例

例：テスト緑・evalスコア基準超え・コンフォーマンス監査クリア・重大警告ゼロ、を必須条件にする。

P POINT

再確認

結論：リリースは“イベント”ではなく“条件の充足”。ゲートがFDEの判断を支えます。

品質ゲート：条件を満たして初めて“リリース可”

✓ 全テストが緑（pass）

✓ eval スコアが基準超え

✓ コンフォーマンス監査クリア

✓ 重大警告ゼロ・証跡あり

品質
ゲート

✓ 全条件クリア
→ リリース可

✕ 条件を一つでも欠けば → 差し戻し（リリース不可）

監査でよくある落とし穴

監査を形だけにすると、証跡はあるのに品質が守られない“なんちゃって監査”になります。

- テストは緑だが、テスト自体が薄い（カバレッジの錯覚）
- 監査差分を出しても、誰も読まず承認だけ押す
- メトリクスを集めるが、閾値（ゲート）が決まっていない
- 証跡が散在し、後から追えない
- 対策：ゲートを少数・明確にし、必ず人が要所を読む

勘でリリース と ゲートでリリース

× 勘でリリース

- 「たぶん大丈夫」で本番反映
- 判断基準が人によって違う
- 問題発生時に原因を追えない
- → 信頼を一度で失う

○ ゲートでリリース

- 明文化した条件を全て満たして反映
- 誰が見ても同じ判断になる
- 証跡で原因をすぐ追える
- → 自動化を安心して広げられる

SECTION 6 — まとめ

“出して良いか”を、勘ではなく証拠で。

仕様照合・監査証跡・メトリクス・品質ゲート。ここがFDEの信頼を生みます。次はリリースと運用へ。

07

SECTION

リリースと運用：剥がせる設計

本番へ安全に出し、運用しながら改善し、モデル進化で壊れないハーネスを保ちます。

リリースと運用フェーズ

P POINT

結論

リリースはゴールではなくスタート。出した後に“安全に運用し続ける”仕組みまで含めて設計します。

R REASON

理由

本番では想定外が必ず起きます。デプロイの安全策と、運用中の観測・改善ループがないと事故が放置されるからです。

E EXAMPLE

具体例

段階的リリース（カナリア等）、ロールバック手段、運用ログの監視、定期的なハーネス見直しをセットで用意します。

P POINT

再確認

結論：FDEの責任は“出すまで”ではなく“成果が安定するまで”。運用設計まで含めて一気通貫です。

CI連携と本番デプロイのガードレール

リリースの瞬間こそ、最も強いガードレールを置く場所です。

- CIで品質ゲートを強制（緑でなければマージ・デプロイ不可）
- 段階的リリース（少数→全体）で影響範囲を絞る
- ワンクリックで戻せるロールバック経路を用意
- 破壊的操作・本番反映は人間の最終承認を必須に
- 原則：戻せる範囲でしか、前に進めない

rippable harness : モデル改善で壊れない設計

P POINT

結論

ハーネスは“いつでも剥がせる”状態に保ちます。凝りすぎた仕組みは、モデルが賢くなると足かせになります。

R REASON

理由

モデルは急速に進化し、昨日必要だった補助が今日は不要になります。固い作り込みは更新のたびに負債化するからです。

E EXAMPLE

具体例

複雑なミドルウェアより、良いAGENTS.mdとpre-commitフック。捨てやすい単純な部品の組み合わせで構成します。

P POINT

再確認

結論：強いハーネスとは“凝った”ではなく“剥がせる”もの。シンプルさが将来の柔軟性を守ります。

過剰設計の兆候と対処

1 誰も触れない複雑さ

→ 単純な部品に分解し、捨てられる形にする

2 モデル更新で毎回壊れる

→ モデル依存の作り込みを薄くする

3 使われない制約

→ 痛みが出ていない制約は思い切って外す

運用後の継続改善ループ

運用で得た“現実の失敗”こそ、ハーネスを鍛える最良の教材です。

- 本番で起きた失敗ごとに、再発防止策を1つ足す
- メトリクスを定点観測し、悪化の兆候を早く掴む
- 不要になった制約・仕組みを定期的に棚卸しする
- 学びを AGENTS.md / 仕様 に還元し、次に活かす
- 原則：ハーネスは“育てる”もの。完成形はない

可観測性を運用に効かせる

P POINT

結論

運用フェーズでこそ、可観測性（柱5）が真価を発揮します。

R REASON

理由

問題は本番で起きます。ログ・メトリクス・証跡が揃っていれば、再現・原因特定・改善が回せるからです。

E EXAMPLE

具体例

異常検知のアラート、コストの監視、実行ログの保全。これらが次の改善（再発防止テスト）の入口になります。

P POINT

再確認

結論：観測→学習→補強→また観測。可観測性が継続改善ループを駆動します。

作り込みすぎ と 剥がせる設計

× 作り込みすぎ

- 複雑なミドルウェアに依存
- モデル更新のたびに壊れる
- 誰も全体を理解できない
- → 改善が止まり、負債になる

○ 剥がせる設計

- AGENTS.md + フック等の単純な部品
- 不要になれば即外せる
- 新しいモデルにすぐ乗り換えられる
- → 長く使える、育てられる

SECTION 7 — まとめ

出して終わりではなく、育て続ける。

段階リリース・ロールバック・継続改善・剥がせる設計。いよいよ全工程を通すCapstoneへ。



SECTION

Capstone：要件1枚からリリースまで完走

ここまでのハーネスを総動員し、小さな実プロジェクトを要件からリリースまで一気通貫で通します。

Capstone 前半：要件 → 仕様 → 設計

ゴール：1枚の要望から、仕様・制約・設計までをエージェントと固める（Section 2～3の総合）

Claude Code

- ① 要件
 - > 要望から `spec.md` を作成
(受け入れ基準を明確に)
- ② 制約
CLAUDE.md に制約を記述
- ③ 設計
 - > 制約準拠の設計案 +
リスク/代替案を提示して

Codex CLI

- ① 要件
 - > `requirements→spec.md`
- ② 制約
AGENTS.md に規約を記述
- ③ 設計
 - \$ `codex`
 - > 設計を提案し理由を説明
(承認まで実装しない)

Antigravity CLI

- ① 要件
 - > `/goal` 要望を仕様化
- ② 制約
AGENTS.md に制約を追記
- ③ 設計
 - > `/goal` 制約準拠の設計
並列レビューで矛盾点検

Capstone 後半：開発 → テスト → 監査 → リリース

ゴール：段階実装・自動テスト・コンフォーマンス監査・ゲート通過まで完走する（Section 4～7の総合）

Claude Code

- ④ 開発
 - > 計画→承認→1ステップ実装
- ⑤ テスト
 - > テスト&evalを生成・実行
- ⑥ 監査
 - > 仕様と実装の差分を監査
- ⑦ リリース
 - ゲート通過を確認しタグ付け

Codex CLI

- ④ 開発
 - > 段階実装（diff確認）
- ⑤ テスト
 - \$ `codex exec "test+eval"`
- ⑥ 監査
 - > 仕様適合をレポート
- ⑦ リリース
 - CIゲート緑を確認

Antigravity CLI

- ④ 開発
 - > 1ステップずつ実装
- ⑤ テスト
 - > `/schedule eval`を定期実行
- ⑥ 監査
 - > `/goal` 仕様適合を監査
- ⑦ リリース
 - ゲート充足を確認

課題テーマの例（小さく完走できるもの）

“完走できる小ささ”が大切。1～2日で要件からリリースまで通せる規模を選びます。

- URL短縮API（作成・リダイレクト・失敗時の挙動を仕様化）
- TODOアプリのバックエンド（CRUD＋バリデーション＋異常系）
- CSV取り込みバリデータ（不正行の検出と監査ログ）
- 簡単なRAG検索エンドポイント（評価セットでevalを回す）
- ポイント：受け入れ基準とゲートを最初に決めてから着手する

評価ループリック（自己採点の観点）

1 仕様の質

受け入れ基準が検証可能で、スコープが明確か

2 制御の質

計画→承認→段階実装で、黒箱を避けられたか

3 検証の質

テスト/evalが基準を網羅し、自動で回るか

4 監査と判断

差分監査・証跡・ゲートでリリース可否を説明できるか

Capstone — まとめ

要件1枚から、説明できるリリースまで。

“動いた”ではなく“安全に出せると説明できる”。これがFDEとしての完走です。

学んだことをFDEのキャリアへ接続する

P POINT

結論

ハーネス設計力は、そのままFDE（Forward Deployed Engineer）の中核スキルになります。

R REASON

理由

FDEは現場で“成果が安定するまで”を担う役割。要件からリリース・運用まで一気通貫で回せる人材が求められているからです。

E EXAMPLE

具体例

本講座で組んだCapstoneは、面接やポートフォリオで「自動化を安全に設計・運用できる」証拠として示せます。

P POINT

再確認

結論：ハーネスを語れることが、FDEとしての説得力になる。技術 × ビジネス × 現場推進の核がここにあります。

全体復習：8つのフェーズ

0 オリエンテーション

FDEとハーネスの全体像 / 5要素

1 原則（5本柱）

コンテキスト・制約・検証・メモリ・可観測性

2-3 要件 → 設計

仕様駆動 / 制約を先に渡す

4-5 開発 → テスト

委譲とガードレール / test + eval

6-7 監査 → リリース

ゲートで判断 / 剥がせる設計で運用

次の一歩：学びを止めないために

ハーネスは“育てるもの”。講座後も小さく実践を続けることで、スキルは定着します。

- 自分の業務タスクを1つ、要件→リリースのハーネスで回してみる
- Capstoneを少しずつ拡張し、再発防止テストを足し続ける
- 3ツールの公式ドキュメント・変更履歴を定期的に追う
- メトリクスを実測し、どのハーネス層に投資すべきか判断する
- Spec-Driven Developmentの各社実装（Spec Kit / Kiro / BMAD等）を触る

THANK YOU

目指せFDE。 仕組みで、信頼をつくる。

AIエージェントを“作る人”から、“安全に率いる人”へ。

あなたのハーネス設計が、現場の成果を支えます。